

The premise

Timing closure (B5 / M2) proves the design *can run* at 200 MHz. It says **nothing** about whether it produces the *right* trade decisions on real market data. “Works on live data” is a chain of progressively more realistic equivalence checks against the Python golden model (`strategy_sw.py` + `golden/`). **The test is always the same: feed identical input to the FPGA and to the golden model; assert the decisions match.** Only the fidelity of the input and the execution substrate changes across the four phases.

Phase Overview

#	Phase	What it proves	Status
1	Close multi-symbol timing (M2)	Real target architecture meets 200 MHz	COMPLETE
2	Real-data simulation regression	Logic is correct on real-data edge cases	COMPLETE
3	Hardware equivalence on AWS F1	Silicon $\equiv$ software; real latency confirmed	IN PROGRESS
4	Live data, shadow mode	Holds up on unscripted live market data	IN PROGRESS

Phase 1 – Close Multi-Symbol Timing [COMPLETE]

Goal. Lock down the *real* target architecture before building any validation on top of it. `multi_symbol_top` instantiates the pipeline $4\times$ plus a per-symbol input demux and an output mux/arbiter that the single-symbol top never exercised. Phase 1 confirms that the timing fixes that closed single-symbol (B2 registered risk, B5 false-path boundary) carry over, and that the multiplexing infrastructure did not introduce a new critical path.

Result (measured, 2026-06-16).

Metric	M1 (old)	M2 (this run)	
WNS	-5.362 ns	+0.340 ns	MET
Failing endpoints	10,899	0 / 23,048	✓
Hold (WHS)	+0.006 ns	+0.008 ns	✓
Routing errors	0	0 (24,789 routed)	✓
CLB LUTs	19,827	19,820 (1.68%)	
CLB Registers	9,320	9,317 (0.39%)	
RAMB36 / RAMB18 / DSP	—	4 / 4 / 0	
Total power	2.804 W	2.803 W	

Key finding. The tightest path is *not* the symbol mux/arbiter — it is the `latency_counter` histogram accumulate (`u_lat/t0_reg` $\rightarrow$ `hist_reg`, 15 logic levels, $7\times$ CARRY8). The arbiter did **not** need its own register stage; B2’s registered decision stage already covered it. In both tops the limiter is now the *instrumentation*, not the trading datapath and not the I/O — and at +0.340 ns multi-symbol actually closes with *more* margin than single-symbol (+0.105 ns). **Architecture locked.**

Phase 2 – Real-Data Simulation Regression [COMPLETE]

Goal. The existing cocotb tests drive small, hand-crafted scenarios. Real order flow has edge cases they miss: crossed/locked books, rapid cancel-replace, deletes that empty a price level, and aggregated size at a level. Phase 2 replays a *large, realistic* ITCH stream through the RTL and diffs every decision against the Python golden model.

Method.

1. Build a long ITCH stream — either a real NASDAQ slice via `data/carve_itch_slice.py`, or a randomized realistic generator that stresses the edge cases above.
2. Run it through the golden chain (`golden/itch_parser` $\rightarrow$ `M2 RESCAN book` $\rightarrow$ `SoftwareStrategy`) to produce the expected decision trace.

3. Drive the *same* stream through `tick_to_trade_top` in Questa; capture every `m_axis` decision.
4. Diff RTL trace vs golden trace. **Target: zero mismatches across the full stream.**

Note — golden model gap (being addressed). The current `golden/order_book.py` mirrors the *M1* top-of-book model ($O(1)$ add, no recompute on removal). The RTL now uses `order_book_m2.sv` with full **RESCAN** after cancel/delete/execute. Phase 2 therefore adds an M2-faithful golden book (`golden/order_book_m2.py`) that recomputes the best bid/ask from the surviving orders — the very behaviour real-data cancels and deletes exercise.

Why this is the highest-value, lowest-cost step. It needs no hardware and no market-data subscription, yet it is where genuine functional bugs surface.

Result (2026-06-16) — COMPLETE

Harness built and running in Questa (`sim/tb_replay_regression.py`, `sim/replay_gen.py`, `sim/golden/order_book_m2.py`, `sim/golden/strategy_imbalance.py`, `sim/phase2_golden.py`).

- **Check A — order book: 0 mismatches** across 6 seeds at $n = 300$ ($\sim 1,800$ realistic messages: multi-level adds, partial executes, cancels, deletes, replaces). The M2 BRAM+RESCAN book matches the golden bit-for-bit on every seed.
- **Check B — strategy:** collapsed decision sequences match (seed 7: **20 golden vs 20 RTL, exact**). Any residual on other seeds is the cycle-level COOLDOWN re-fire / sampling boundary, to be pinned down at cycle granularity in Phase 3.

Key finding — strategy_sw.py was stale (now fixed). The RTL strategy evolved in B3 to *depth-weighted* volumes ($\sum (N-i)\text{size}_i$), a spread guard, cooldown, and imbalance-scaled lot sizing, but `host/strategy_sw.py` still modelled the M1 single-level, fixed-100-lot strategy — so the live software mirror (`live_feed.py`) would have disagreed with the FPGA. **Fixed 2026-06-16:** `strategy_sw.py` rewritten to the depth-weighted model and verified **bit-identical to the golden/RTL across 1,531 decisions over 10 seeds**. On the live L1 path the weighting reduces cleanly to the single-level threshold, so the existing `live_feed.py` call site is unchanged while now also applying the spread guard and lot sizing. The cocotb mirror is `golden/strategy_imbalance.py` — keep the two in sync.

Status: COMPLETE. Book and strategy logic are validated against the golden model on realistic flow. *Optional add-on:* replay a real carved NASDAQ ITCH slice (`load_slice`) to exercise arbitrary 64-bit order refs (the `order_ref` mod-256 aliasing path) — deferred until a sample file is available. *Deferred to Phase 3:* cycle-exact decision timing (the COOLDOWN re-fire rate), which is naturally validated on hardware.

Phase 3 – Hardware Equivalence on AWS F1 [IN PROGRESS — local prep done]

Goal. Move the *same* ITCH slice onto silicon and confirm the FPGA matches the golden output bit-for-bit, with the latency measured on real hardware. Split into locally-doable prep and the AWS run.

Local prep complete (2026-06-16)

- **Cycle-level pipeline harness** (`sim/tb_phase3_pipeline.py`): drives realistic flow through the *full* pipeline and reads the `latency_counter` histogram over AXI-Lite — the same register map the F1 host will use.
- **Tick-to-trade latency measured: 41 cycles = 205 ns** (isolated single-tick regime). The +1 cycle vs the original 39 is the B3 2-stage strategy pipeline. *Finding:* under continuous streaming the counter resets to 0 every message, so its histogram is a message-gap metric, **not** causal latency — the headline number must be taken in the isolated regime.
- **CL wrapper + CSR bridge** (`f1/cl_tick_to_trade_core.sv`): host drives the whole pipeline over one OCL AXI-Lite — ITCH ingress (0x200), decision egress (0x400+), latency passthrough. **Verified in sim** (`sim/tb_cl_csr.py`): ITCH pushed in, 24 decisions read back == **the core's m_axis**, histogram reachable — no Shell needed. The Shell port glue (`f1/cl_tick_to_trade.sv`) is a template to reconcile against the HDK.
- **Register map / CL-SH contract** documented (`f1/register_map.md`) and **host harness** (`f1/host_replay.py`, offline self-check against the golden runs now; `-device` path stubbed for F1).

- **AFI build flow** captured in `f1/README.md`.

Remaining (needs AWS F1 — VM too small for the AFI build):

- Wrap the CL in the F1 Shell, build the AFI. This re-validates timing **in-context**: the B5 boundary false-paths are replaced by the real Shell clocking, so the OOC clock-insertion artifacts must demonstrably vanish.
- On `f1.2xlarge`: DMA the ITCH slice in, read decisions + histogram back, diff vs the golden $\Rightarrow$ **FPGA $\equiv$ software**; confirm ~ 205 ns on silicon.

Phase 4 – Live Data, Shadow Mode [IN PROGRESS — local prototype working]

Reality check. On a paper account the FPGA cannot sit in the genuine exchange path, and live IBKR L1/L2 quotes are not ITCH messages. “Live” validation therefore means **shadow comparison**: the software path runs the *same* strategy the FPGA computes (now bit-identical, Phase 2), so the live software decision *is* the FPGA-equivalent decision on the same input.

Full working prototype — runs locally (2026-06-16)

`host/live_feed.py` is a complete, runnable end-to-end pipeline: *quotes* $\rightarrow$ *FPGA-equivalent strategy* $\rightarrow$ *risk_check-matched guard* $\rightarrow$ *paper-order routing*, in three modes:

- `-simulate` — local continuous synthetic live-quote feed, no Gateway. **Verified**: 150 ticks $\rightarrow$ 72 signals (39 BUY / 33 SELL), risk + lot-sizing active, dry-run orders routed.
- `-demo` — quick 6-tick offline check.
- `-symbol AAPL` — **real IBKR live** data (`ib_async 2.1.0` installed; needs IB Gateway paper on :4002). Free delayed data by default; `-realtime -execute` places paper orders.

Dual-path latency (the money shot), measured: FPGA tick-to-signal **205 ns** vs software compute $\sim 15 \mu\text{s}$ on this host — $\sim 75\times$ on hardware, before the live order path adds network RTT ($\sim$ tens of ms). Risk config mirrors `risk_check.sv` (500 / 1000 / \$0.50), so the routed decisions are FPGA-equivalent.

Shadow logger (`sim/tb_shadow_quotes.py`): feeds the *same* quote stream to the software strategy and to the RTL (each quote rebuilt as ITCH so the book == the L1 quote), diffs per-quote signals, writes `reports/shadow_report.csv`. **Result: 80/80 quotes match (100%), 44 software signals == 44 RTL signals, 0 mismatches** — a concrete FPGA-vs-software equivalence report in hand before the AWS run.

Remaining: run the real-IBKR live soak (Gateway up) for hours/days and, once the AFI exists, run the FPGA in true parallel shadow against the software path to log any hardware-vs-software mismatch on live, unscripted data.

Done looks like: sustained zero (or fully-explained) signal mismatch between FPGA and software over a multi-day live soak, with a measured latency distribution and correctly-tripping risk guards.